

Cloud Distributed Applications

Realtime Analytics Dashboard

Ciorăţanu Maria, Pâncă Aida-Gabriela, Varzar Alina-Miruna

Faculty of Computer Science, Master of Software Engineering, Year 1, Group 1

April 2026

Contents

1	Introduction	3
2	System Architecture	3
2.1	General Architecture	3
2.2	Cloud-Native Services	4
2.3	Main Components	4
3	Implementation	6
3.1	Event Publishing from Service A	6
3.2	Cloud Function Processing	6
3.3	WebSocket Gateway Runtime Metrics	7
3.4	Backpressure Through Coalescing	7
3.5	WebSocket Reconnection	7
3.6	Deployment and Verification Scripts	7
4	Usage	8
5	Communication Analysis	10
5.1	Synchronous REST Communication	10
5.2	Asynchronous Pub/Sub Communication	10
5.3	Realtime WebSocket Communication	11
5.4	Internal gRPC Communication	11
6	Consistency Analysis	12
6.1	Eventual Consistency Model	12
6.2	Idempotency and At-Least-Once Delivery	12
6.3	CAP Trade-Offs	12
7	Experimental Methodology	12
7.1	Test Environment	12
7.2	Collected Metrics	13
7.3	Benchmark Scenarios	13

8	Experimental Results	14
8.1	Variable-Volume Benchmark	14
8.2	Consistency Window Benchmark	15
8.3	Concurrency Benchmark	15
8.4	Reliability Summary	16
9	Graphs	17
9.1	REST Throughput Under Concurrency	17
9.2	Latency Percentiles Under Concurrency	17
9.3	Backpressure Behavior	18
9.4	Variable-Volume Latency	18
10	Analysis of the Experimental Results	19
10.1	Throughput and Completion Rate	19
10.2	Latency Behavior	19
10.3	Backpressure and WebSocket Scalability	19
10.4	Consistency Window	19
10.5	Bottlenecks	20
11	Scalability Discussion	20
12	Resilience Analysis	20
12.1	Decoupling Through Pub/Sub	20
12.2	Idempotent Event Processing	20
12.3	gRPC Fallback	21
12.4	WebSocket Reconnection	21
12.5	Protected Debug Endpoints	21
13	Comparison with a Real System	21
14	Conclusions	23

1 Introduction

This report presents the design, implementation and experimental evaluation of a distributed cloud application developed for the **Concurrent and Distributed Programming** course. The chosen assignment was **Project 1 - Real-Time Analytics Dashboard**. The objective was to extend an existing REST API application into a cloud-native distributed system that collects resource access events, processes them asynchronously and displays aggregated analytics in real time.

The base application used in this project is **Fast Lazy Bee**, a movie REST API implemented with Node.js, Fastify and MongoDB. In the original application, the client accesses movie resources through HTTP endpoints. In our implementation, each access to a movie resource can also generate an analytics event. This event is not processed inside the main request path. Instead, it is sent to an asynchronous event pipeline built with Google Cloud services.

The final system uses **Google Cloud Run**, **Google Cloud Functions Gen2**, **Google Pub/Sub**, **Google Firestore**, **Google Cloud Build**, **Google Artifact Registry**, and **MongoDB Atlas**. The system also includes a WebSocket Gateway and a separate dashboard client for real-time updates. In addition, a gRPC Analytics Service was implemented for internal service-to-service communication between the WebSocket Gateway and the analytics storage layer.

The project is built around several important distributed systems concepts studied in the course and laboratories: microservices, event-driven communication, asynchronous processing, eventual consistency, cloud-native services, fault tolerance, observability and scalability. The report focuses not only on the final implementation, but also on the reasoning behind the architectural choices and on the interpretation of the performance results.

2 System Architecture

2.1 General Architecture

The system follows an event-driven architecture. The base API receives the client request synchronously, while the analytics processing is executed asynchronously through Pub/Sub and a Cloud Function. The real-time dashboard is updated through a WebSocket connection.

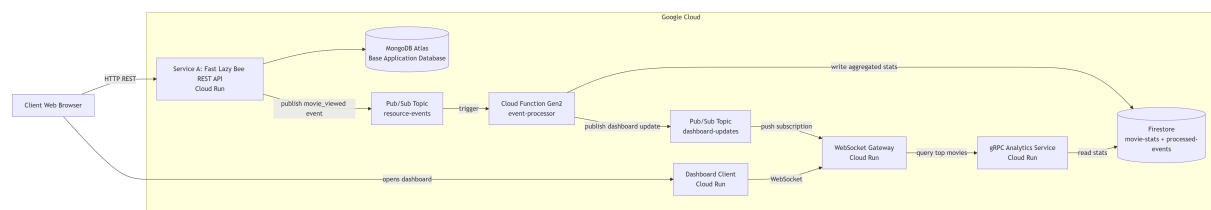


Figure 1: Implemented distributed architecture for the realtime analytics dashboard

The main data flow is the following:

1. A client accesses a movie resource through the Fast Lazy Bee REST API.

2. Service A returns the movie response and publishes a `movie_viewed` event to Pub/Sub.
3. The Cloud Function is triggered by the event and updates aggregated statistics in Firestore.
4. The Cloud Function publishes a processed dashboard update to a second Pub/Sub topic.
5. The WebSocket Gateway receives the update through a push subscription.
6. The dashboard client receives the update in real time through WebSocket.

2.2 Cloud-Native Services

The project uses Google Cloud as the selected cloud provider. The architecture is built around managed cloud services that separate deployment, event delivery, serverless processing, persistent state and real-time communication. This allows each part of the system to have a clear responsibility and to be deployed or verified independently.

Table 1: Cloud-native services used in the project

Cloud service	Role in the system
Google Cloud Run	Hosts the Fast Lazy Bee API, WebSocket Gateway, Dashboard Client and gRPC Analytics Service as independently deployed services
Google Cloud Functions Gen2	Provides the FaaS component used for asynchronous event processing
Google Pub/Sub	Provides the event broker for asynchronous communication between services
Google Firestore	Stores the aggregated analytics state and the processed event identifiers used for idempotency
Google Cloud Build	Builds container images for Cloud Run deployments
Google Artifact Registry	Stores the container images used by Cloud Run
MongoDB Atlas	Stores the base application data used by Fast Lazy Bee

Firestore is used as the analytics state store, while MongoDB Atlas remains the operational database of the base movie API. This separation keeps the original application data independent from the derived analytics data.

2.3 Main Components

Service A - Fast Lazy Bee

Service A is the base REST API application. It exposes movie endpoints and retrieves movie data from MongoDB Atlas. The endpoint `GET /api/v1/movies/:movie_id` was extended so that a successful movie access also publishes a `movie_viewed` event to Pub/Sub.

The event publishing logic is controlled through environment variables. This makes the feature easy to enable in the cloud deployment and to disable during local development if needed.

Pub/Sub Topics

The system uses two Pub/Sub topics:

- **resource-events**: receives raw movie access events from Service A
- **dashboard-updates**: receives processed analytics updates from the Cloud Function

The separation between the two topics keeps the raw event stream and the dashboard update stream independent.

Cloud Function - Event Processor

The Cloud Function is the **FaaS** component of the project. It is triggered by messages from **resource-events**. It decodes the event, updates the aggregate document for the movie in Firestore and records the processed message identifier in the **processed-events** collection.

This design is important because Pub/Sub provides at-least-once delivery. Therefore, the same message can theoretically be delivered more than once. The **processed-events** collection is used to make the function idempotent and to avoid counting the same message twice.

Firestore Analytics Store

Firestore stores the analytics state. The main collection is **movie-stats**, where each document contains the movie identifier, movie title, view count and timestamps for the latest processed event. A second collection, **processed-events**, is used for idempotency. The analytics state is eventually consistent with the access events. The REST API response is not blocked by the analytics processing.

WebSocket Gateway

The WebSocket Gateway is deployed as an independent Cloud Run service. It receives dashboard updates from Pub/Sub through a push endpoint, computes runtime metrics and broadcasts updates to connected dashboard clients.

The gateway also implements a **backpressure** mechanism. When many updates arrive quickly, it coalesces multiple updates and sends fewer WebSocket broadcasts. This reduces pressure on connected clients while still keeping the dashboard state up to date.

gRPC Analytics Service

The gRPC Analytics Service is another independent Cloud Run service. It exposes a small internal API used by the WebSocket Gateway to retrieve top viewed movies. This service reads data from Firestore and returns the results through gRPC.

The gateway uses gRPC as the primary path for top movie queries and can fall back to Firestore directly if the gRPC service is unavailable. This improves resilience while also demonstrating internal service-to-service communication.

Dashboard Client

The dashboard is deployed as a separate Cloud Run service and is implemented using HTML, CSS and JavaScript. It connects to the WebSocket Gateway, receives real-time updates and displays:

- connected WebSocket clients
- total analytics updates
- total WebSocket broadcasts
- coalesced updates
- top viewed movies
- recent activity
- latest processed event
- p50, p95 and p99 latency values
- real-time latency chart

The dashboard also reconnects automatically when the WebSocket connection is closed.

3 Implementation

3.1 Event Publishing from Service A

The movie endpoint was extended so that every valid movie access generates an event. The event contains a generated event identifier, the movie identifier, the movie title, the access timestamp and the source service. The payload is serialized as JSON and published to the Pub/Sub topic `resource-events`.

The implementation keeps the analytics logic outside the normal response generation. The REST request can return the movie resource even though the analytics update is still being processed in the background. Analytics processing should not increase the user-facing latency of the movie API, so it is executed asynchronously after the movie response is prepared.

3.2 Cloud Function Processing

The Cloud Function performs the following steps for each received Pub/Sub message:

1. decode the Pub/Sub payload
2. extract or generate a stable message identifier
3. check the `processed-events` collection

4. update or create the corresponding document in `movie-stats`
5. store the processed message identifier
6. publish the processed dashboard update to `dashboard-updates`

The update to Firestore is performed in a transaction. This ensures that the idempotency check and the statistics update are handled consistently.

3.3 WebSocket Gateway Runtime Metrics

The WebSocket Gateway maintains runtime metrics in memory. These metrics are used by the dashboard and by the benchmark scripts. The most important metrics are:

- `totalUpdates`: number of processed updates received by the gateway
- `totalBroadcasts`: number of WebSocket broadcasts sent
- `coalescedUpdates`: number of updates merged by the backpressure mechanism
- `latestLatencyMs`: latest end-to-end latency sample
- `p50LatencyMs`, `p95LatencyMs`, `p99LatencyMs`: latency percentiles
- `connectedClients`: number of active dashboard connections

The gateway exposes these values through `/metrics` and `/snapshot`. The dashboard reads an initial snapshot and then continues receiving live updates through WebSocket.

3.4 Backpressure Through Coalescing

The backpressure mechanism is implemented in the WebSocket Gateway. Instead of broadcasting every update immediately, the gateway can keep the latest pending update and broadcast at a controlled interval. If more updates arrive before the broadcast happens, they are coalesced.

This is useful during traffic bursts. A dashboard does not need to receive every intermediate state if the latest state already contains the updated counters and top movies. Therefore, coalescing reduces the number of WebSocket messages without losing the final aggregated state.

3.5 WebSocket Reconnection

The dashboard client includes automatic WebSocket reconnection logic. If the connection is interrupted, the client waits for a short interval and then tries to reconnect to the gateway. This behavior was tested by using a protected debug endpoint that closes the active WebSocket clients. After the interruption, the dashboard restored the connection and continued receiving live updates.

3.6 Deployment and Verification Scripts

The repository includes PowerShell scripts for verification and benchmarking:

- `verify-cloud-deployment.ps1`: checks all required cloud components and environment variables

- `verify-grpc-analytics.ps1`: validates the gRPC path from the gateway to the analytics service
- `smoke-test.ps1`: validates the full end-to-end flow
- `benchmark-burst.ps1`: sends a fixed number of requests and measures backpressure and latency
- `benchmark-load-series.ps1`: runs multiple request volumes
- `benchmark-consistency.ps1`: measures the consistency window
- `benchmark-concurrency.ps1`: uses `hey` to test concurrent traffic

These scripts made the testing reproducible and helped collect CSV and JSON benchmark results.

4 Usage

This section presents the main commands used to build, verify and benchmark the implemented system. The project was developed and tested mainly from Windows PowerShell, while the deployed services run in Google Cloud. Unless specified otherwise, the commands are executed from the repository root.

Local Build

The Fast Lazy Bee application is the TypeScript service extended with event publishing. The local build can be checked with:

```
npm install
npm run build
```

A successful build confirms that the modified TypeScript source code compiles correctly, including the Pub/Sub event publishing logic added to the movie endpoint.

Common PowerShell Variables

Most verification and benchmark scripts use the same Google Cloud region, movie identifier and debug token. The following variables were used during testing:

```
$REGION="us-central1"; $MOVIE_ID="573a1390f29313caabcd42e8"; $env:
DEBUG_TOKEN="<configured-debug-token>"
```

The service URLs can be read directly from Cloud Run:

```
$FAST_URL=(gcloud run services describe fast-lazy-bee --region $REGION --
format="value(status.url)"); $WS_GATEWAY_URL=(gcloud run services
describe websocket-gateway --region $REGION --format="value(status.url)")
; $DASHBOARD_URL=(gcloud run services describe dashboard-client --region
$REGION --format="value(status.url)")
```

These variables are used for manual tests, smoke tests and benchmark scripts.

Cloud Deployment Verification

After deployment, the complete cloud setup can be verified using:

```
powershell.exe -ExecutionPolicy Bypass -File .\scripts\verify-cloud-  
deployment.ps1 -Region $REGION
```

This script checks the Cloud Run services, the Cloud Function, Pub/Sub topics and subscriptions, environment variables, health endpoints, dashboard configuration and the gRPC integration state.

The gRPC path can also be verified separately:

```
powershell.exe -ExecutionPolicy Bypass -File .\scripts\verify-grpc-analytics  
.ps1 -Region $REGION
```

This validates the internal communication path:

```
websocket-gateway -> gRPC -> grpc-analytics-service -> Firestore
```

Smoke Test and Manual End-to-End Test

A complete end-to-end smoke test can be executed with:

```
powershell.exe -ExecutionPolicy Bypass -File .\scripts\smoke-test.ps1 -  
Region $REGION -MovieId $MOVIE_ID -WaitSeconds 90
```

The smoke test checks the main runtime flow of the system: health endpoints, dashboard deployment, gateway metric reset, movie access event generation and event propagation to the WebSocket Gateway snapshot.

The event pipeline can also be tested manually. First, one movie resource is accessed through Service A:

```
curl.exe -s -H "Cache-Control: no-cache" "$FAST_URL/api/v1/movies/$MOVIE_ID"  
| jq
```

After a short asynchronous processing delay, the WebSocket Gateway snapshot can be inspected:

```
Start-Sleep -Seconds 30; curl.exe -s "$WS_GATEWAY_URL/snapshot" | jq
```

A successful result should show that `metrics.totalUpdates` increased, `lastProcessedUpdate` is populated and the viewed movie appears in the top movies list.

Dashboard Usage

The deployed dashboard can be opened from the Cloud Run URL:

```
Start-Process "$DASHBOARD_URL"
```

The dashboard connects to the WebSocket Gateway using the runtime configuration exposed by the dashboard service. During the demo, movie access events can be generated from PowerShell, and the dashboard updates automatically without refreshing the page. The WebSocket reconnection behavior can be tested by closing connected clients through the protected debug endpoint:

```
curl.exe -s -X POST -H "Content-Type: application/json" -H "x-debug-token: $env:DEBUG_TOKEN" --data "{}" "$WS_GATEWAY_URL/debug/close-clients" | jq
```

After this request, the dashboard client should reconnect automatically.

Benchmark Execution

The burst benchmark sends a fixed number of movie access requests and collects latency, update, broadcast and backpressure metrics:

```
powershell.exe -ExecutionPolicy Bypass -File .\scripts\benchmark-burst.ps1 -  
  Region $REGION -MovieId $MOVIE_ID -Requests 100 -WaitSeconds 30 -  
  OutputDir "benchmark-results"
```

The variable-volume benchmark runs multiple request volumes:

```
powershell.exe -ExecutionPolicy Bypass -File .\scripts\benchmark-load-series  
.ps1 -Region $REGION -MovieId $MOVIE_ID -RequestCounts "10,20,50,100" -  
  WaitSeconds 30 -OutputDir "benchmark-results"
```

The consistency benchmark measures the eventual consistency window:

```
powershell.exe -ExecutionPolicy Bypass -File .\scripts\benchmark-consistency  
.ps1 -Region $REGION -MovieId $MOVIE_ID -Trials 5 -PollIntervalMs 500 -  
  TimeoutSeconds 30 -OutputDir "benchmark-results"
```

The concurrency benchmark uses `hey` to increase the concurrent HTTP load:

```
powershell.exe -ExecutionPolicy Bypass -File .\scripts\benchmark-concurrency  
.ps1 -Region $REGION -MovieId $MOVIE_ID -Requests 100 -ConcurrencyLevels  
  "1,5,10,20" -WaitSeconds 45 -OutputDir "benchmark-results"
```

All benchmark results are saved locally in the `benchmark-results` folder as CSV and JSON files. The numerical values presented in the experimental results section were extracted from these generated files.

5 Communication Analysis

5.1 Synchronous REST Communication

The interaction between the browser or benchmark client and Service A is synchronous HTTP REST. This is appropriate because the client needs an immediate response containing the requested movie resource. REST is simple, easy to test with `curl` and fits the original Fast Lazy Bee API.

However, the analytics processing is not done synchronously. If Service A waited for all analytics components to finish, the user-facing response time would depend on Pub/Sub, Cloud Function, Firestore and the WebSocket Gateway. That would make the movie API slower and more fragile.

5.2 Asynchronous Pub/Sub Communication

The communication between Service A and the event processor is asynchronous through Pub/Sub. This choice decouples the base API from the analytics pipeline. Service A only

publishes an event and does not need to know when or how the event will be processed. The same idea is used between the Cloud Function and the WebSocket Gateway. The Cloud Function publishes processed dashboard events to a second Pub/Sub topic. The gateway receives those events through a push subscription.

This design improves scalability and fault tolerance. If the analytics processor is slower or temporarily unavailable, the event broker can buffer messages and retry delivery.

5.3 Realtime WebSocket Communication

The dashboard uses WebSocket communication because it needs real-time updates without refreshing the page. Unlike normal REST polling, WebSocket keeps a persistent connection open and allows the gateway to push updates immediately to connected clients.

This fits the project requirement for a real-time technology. It also makes the dashboard more useful because changes are visible live.

5.4 Internal gRPC Communication

The gRPC Analytics Service is used for internal communication between the WebSocket Gateway and the analytics service. gRPC is more structured than a simple HTTP endpoint because it uses a schema defined in a `.proto` file. This makes the service contract explicit.

The implemented RPC methods are `Health` and `GetTopMovies`.

The WebSocket Gateway uses gRPC to retrieve top viewed movies from the Analytics Service. If the gRPC call fails, the gateway falls back to Firestore, which keeps the dashboard functional even when the internal analytics service is unavailable.

Table 2: Communication patterns used in the project

Interaction	Communication type	Reason
Client to Service A	Synchronous REST	The client needs the movie response immediately
Service A to Cloud Function	Asynchronous Pub/Sub	Analytics should not block the REST response
Cloud Function to WebSocket Gateway	Asynchronous Pub/Sub push	Processed events are delivered independently to the realtime layer
Dashboard to WebSocket Gateway	WebSocket	The dashboard needs live updates
WebSocket Gateway to gRPC Analytics Service	gRPC	Structured internal service-to-service communication
Services to Firestore	Database read/write	Persistent analytics state and idempotency

6 Consistency Analysis

6.1 Eventual Consistency Model

The system uses an **eventual consistency** model. When a movie is accessed, the REST API response is returned before the dashboard update is visible. The analytics result becomes visible only after the event passes through Pub/Sub, Cloud Function, Firestore and the WebSocket Gateway.

This behavior is intentional. Analytics data does not need to be strictly consistent with the movie API response at the exact same moment. It is acceptable for the dashboard to be updated a short time later, as long as the update is eventually processed correctly.

6.2 Idempotency and At-Least-Once Delivery

Pub/Sub provides at-least-once delivery, which means that duplicate delivery is possible. For this reason, the Cloud Function stores processed message identifiers in the **processed-events** collection. Before updating the movie statistics, the function checks whether the message was already processed.

This makes the event processor idempotent. If the same message is delivered again, the system can skip it instead of increasing the view count twice.

6.3 CAP Trade-Offs

From the CAP theorem perspective, the system favors **availability** and **partition tolerance** for the analytics path over strict immediate consistency. The base REST API can continue serving movie resources even if the analytics update is delayed. Pub/Sub decouples producers and consumers, while Firestore stores the final aggregated state.

The trade-off is that the dashboard can temporarily display slightly older data. This is acceptable for analytics because the dashboard is not used as the source of truth for financial or transactional decisions. The main requirement is that the update is eventually visible and that the system does not lose events under normal operation.

7 Experimental Methodology

7.1 Test Environment

The final tests were executed on the deployed Google Cloud system. The services were deployed in the **us-central1** region. The tested components were:

- **fast-lazy-bee** on Cloud Run
- **event-processor** as a Cloud Function Gen2
- **websocket-gateway** on Cloud Run
- **dashboard-client** on Cloud Run
- **grpc-analytics-service** on Cloud Run
- Firestore collections **movie-stats** and **processed-events**
- Pub/Sub topics **resource-events** and **dashboard-updates**

The benchmark client was executed from Windows PowerShell. HTTP requests were generated with `curl.exe` and `hey`. Results were saved locally as CSV and JSON files in the `benchmark-results` folder.

7.2 Collected Metrics

The following metrics were collected:

- number of successful HTTP requests
- error rate
- REST throughput in requests per second
- total number of processed analytics updates
- processing completion percentage
- number of WebSocket broadcasts
- number of coalesced updates
- end-to-end latency
- p50, p95 and p99 latency
- consistency window
- WebSocket reconnection behavior

The end-to-end latency measures the time from the movie access event to the moment the update reaches the WebSocket Gateway and becomes available for the dashboard.

The consistency window was measured by triggering one event and polling the gateway snapshot until the update became visible.

7.3 Benchmark Scenarios

Three main benchmark scenarios were used:

1. **Variable-volume benchmark:** sequential requests with volumes 10, 20, 50 and 100
2. **Consistency benchmark:** 5 individual trials measuring the consistency window
3. **Concurrency benchmark:** 100 requests sent with concurrency levels 1, 5, 10 and 20 using `hey`

The benchmarks focus on the behavior of the complete distributed pipeline, not only on the REST API response time.

8 Experimental Results

8.1 Variable-Volume Benchmark

The variable-volume benchmark sends sequential movie access requests with increasing total volume.

Table 3: Variable-volume benchmark results

Requests	Success	Failed	Error %	Req/s	Updates	Broadcasts	Coalesced	p50 ms	p95 ms	p99 ms
10	10	0	0	3.76	10	1	9	5066	5893	5893
20	20	0	0	3.89	20	4	16	388	1959	2299
50	50	0	0	3.85	50	10	40	181	1315	1621
100	100	0	0	3.85	100	23	77	183	700	1166

The system processed all generated events successfully. The error rate was 0% for all volumes, and the number of processed updates matched the number of requests in every run.

The first run, with 10 requests, had higher latency. This can be explained by warm-up effects and normal cloud scheduling variability. After that, the measured latency became lower and more stable.



Figure 2: Cloud Function processing completion under variable request volume

In this project, Cloud Function throughput was evaluated indirectly through the number of processed analytics updates during each load scenario. Since each successful movie request generates one Pub/Sub event, the relation between successful requests and `totalUpdates` shows whether the event processor kept up with the input load. In all final benchmark runs, `totalUpdates` matched the number of generated requests, which means that the Cloud Function processed all events within the benchmark waiting window.

8.2 Consistency Window Benchmark

Table 4: Consistency window benchmark results

Metric	Value
Trials	5
Successful trials	5 / 5
Average consistency window	1073.15 ms
Minimum consistency window	1033.68 ms
Maximum consistency window	1154.18 ms
Average end-to-end latency	354.2 ms
Average Cloud Function processing latency	72.2 ms
Average gateway latency	282 ms

The results confirm the eventual consistency behavior. The API response is returned first, and the dashboard update becomes visible shortly afterward. In the final test run, the average measured consistency window was approximately 1.07 seconds.

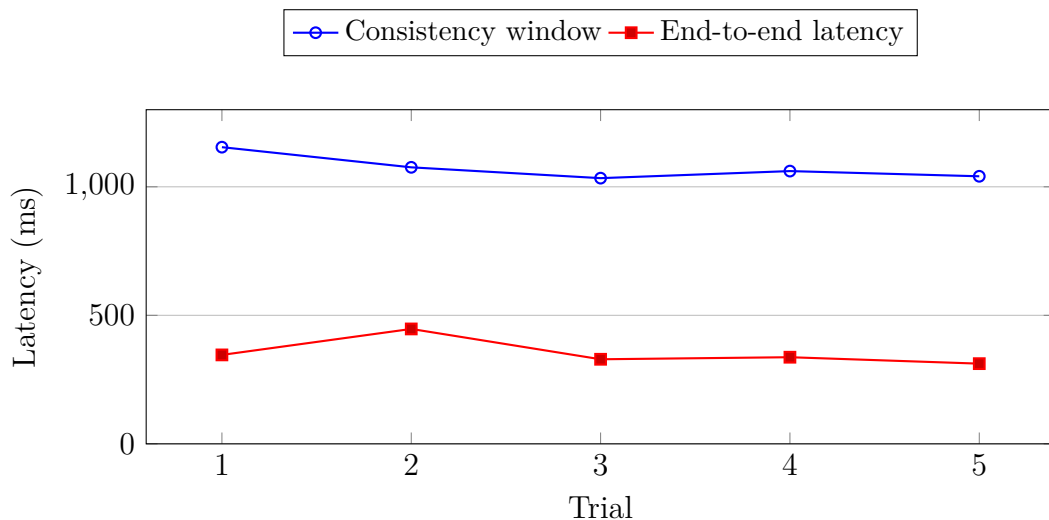


Figure 3: Consistency window and end-to-end latency measured for each individual trial

The individual consistency trials show stable behavior. The measured consistency window stayed between 1033.68 ms and 1154.18 ms, while the end-to-end latency reported by the dashboard update remained lower. This supports the interpretation that the system is eventually consistent and that the dashboard becomes up to date shortly after the REST request is completed.

8.3 Concurrency Benchmark

The concurrency benchmark used `hey` to send 100 requests with increasing concurrency.

Table 5: Concurrency benchmark results

Conc.	HTTP 200	Failed	Error %	REST req/s	Updates	Completion %	Broadcasts	Coalesced	p50 ms	p95 ms	p99 ms
1	100	0	0	6.25	100	100	14	86	258	1396	1921
5	100	0	0	29.38	100	100	11	89	4738	7793	8370
10	100	0	0	58.67	100	100	8	92	3328	6026	6653
20	100	0	0	83.08	100	100	9	91	3250	7160	7724

The REST API remained stable under concurrent access. All requests returned HTTP 200, and all generated events were eventually processed. The REST throughput increased from 6.25 requests/second at concurrency 1 to 83.08 requests/second at concurrency 20.

8.4 Reliability Summary

The benchmark results can also be summarized from the perspective of reliability and processing completion. This is useful because the system is not evaluated only by latency, but also by whether it loses events or produces errors under load.

Table 6: Reliability and processing completion summary

Scenario	Total input	Failed	Error rate	Processed output	Completion
Variable-volume	180 requests	0	0%	180 updates	100%
Concurrency	400 requests	0	0%	400 updates	100%
Consistency	5 trials	0	0%	5 detections	100%

The table shows that the final benchmark runs did not lose generated events. In the variable-volume benchmark, the four runs generated 180 movie access requests in total, and the gateway received 180 processed updates. In the concurrency benchmark, the four runs generated 400 HTTP requests in total, and all corresponding analytics updates were processed. The consistency benchmark also succeeded in all 5 trials. Therefore, the final results show both correct event processing and 0% error rate in the evaluated scenarios.

9 Graphs

9.1 REST Throughput Under Concurrency

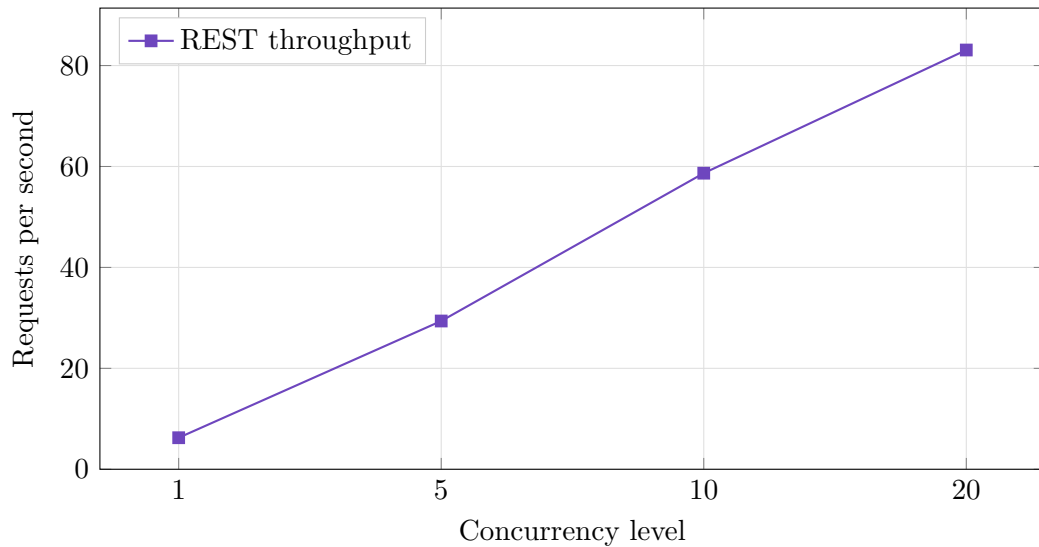


Figure 4: REST throughput for increasing concurrency levels

The graph shows that the REST API scales well when the concurrency level increases. This is expected because Cloud Run can handle concurrent HTTP requests and the movie access endpoint does not wait for the full analytics pipeline to finish.

9.2 Latency Percentiles Under Concurrency

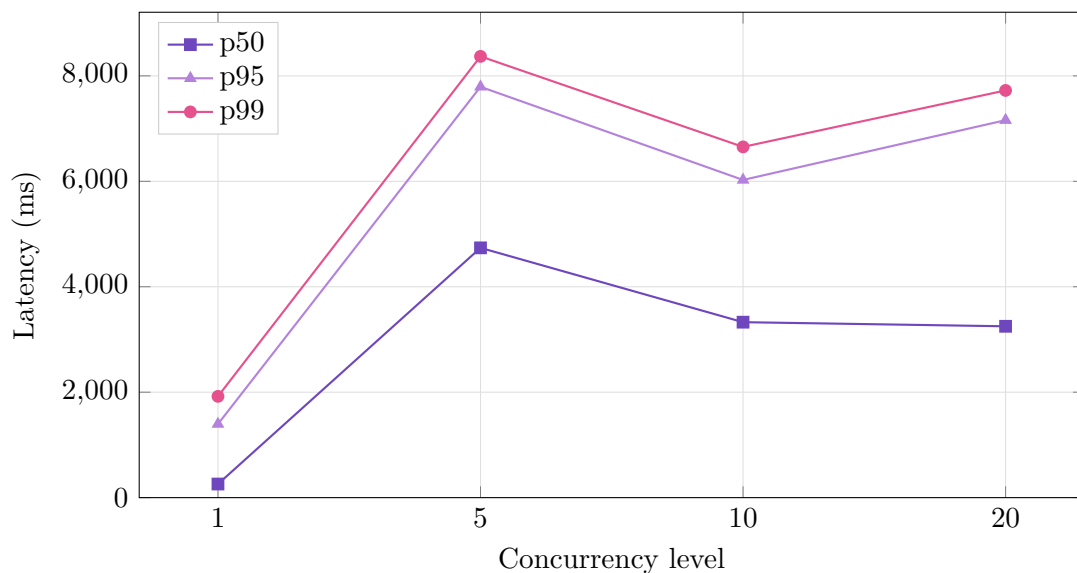


Figure 5: Dashboard latency percentiles under concurrent load

The latency values in this graph represent the asynchronous dashboard pipeline, not only the REST response time. Under higher concurrency, many events enter the analytics

pipeline almost at the same time. This increases dashboard-side latency, even though the REST API remains fast and stable.

9.3 Backpressure Behavior

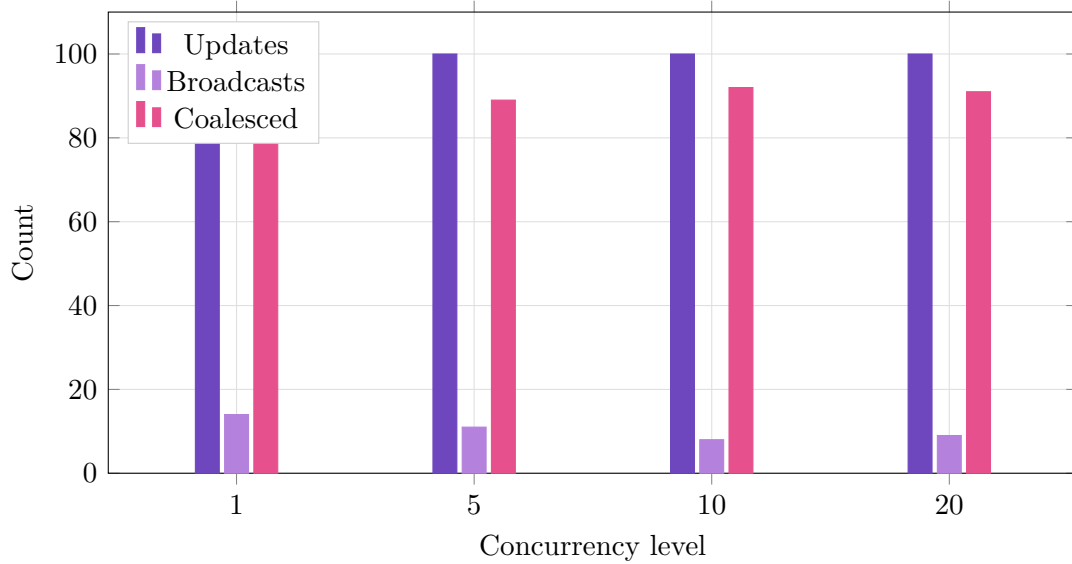


Figure 6: Backpressure effect: updates, broadcasts and coalesced updates

The backpressure mechanism is visible in all concurrency runs. For 100 received updates, the WebSocket Gateway sent far fewer broadcasts. At concurrency 20, only 9 WebSocket broadcasts were sent, while 91 updates were coalesced. This confirms that the gateway reduces pressure on connected clients during bursts.

9.4 Variable-Volume Latency

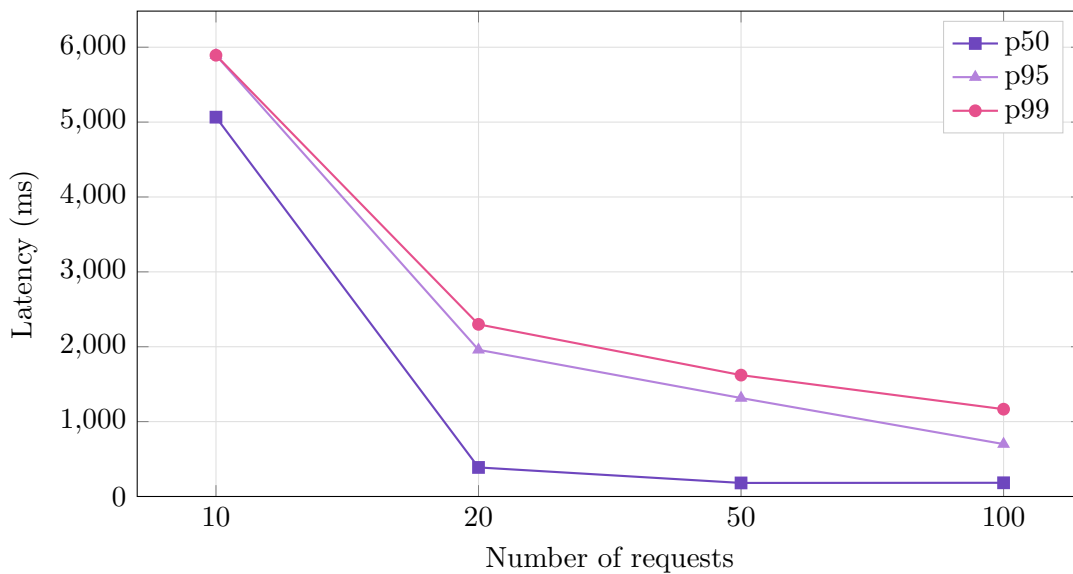


Figure 7: Latency percentiles for increasing request volume

The first run had higher latency percentiles than the larger runs. This is not considered an implementation error, because all requests succeeded and all generated events were processed. The difference is most likely caused by normal variability in the managed cloud pipeline, such as Cloud Run scheduling, Pub/Sub delivery time, Cloud Function execution and Firestore writes. For larger runs, the system became more stable and the percentile values decreased, while still maintaining 100% processing completion.

10 Analysis of the Experimental Results

10.1 Throughput and Completion Rate

The most important positive result is that the system processed all generated events in the final benchmarks. In both the variable-volume benchmark and the concurrency benchmark, the number of processed updates matched the number of successful REST requests.

The concurrency benchmark shows that the REST API can handle a higher request rate when concurrency increases. At concurrency 20, the REST throughput reached 83.08 requests per second, with 0% error rate and 100% event processing completion.

This confirms that Service A is not the bottleneck in the tested scenarios. The movie API can return responses quickly because it does not wait for the analytics pipeline.

10.2 Latency Behavior

The measured dashboard latency is higher than the REST response time because it includes the asynchronous path:

Pub/Sub -> Cloud Function -> Firestore -> Pub/Sub -> WebSocket Gateway

This is expected in an event-driven system. The system trades immediate strict consistency for better decoupling and availability. The dashboard may be updated later, but the REST API remains responsive.

The p95 and p99 values are especially important because they show tail latency. Tail latency increased under concurrent load, which indicates that bursts of events create pressure on the asynchronous pipeline. However, all events were still processed correctly.

10.3 Backpressure and WebSocket Scalability

The backpressure results are strong because the number of WebSocket broadcasts is much smaller than the number of processed updates. This means that the gateway does not push every intermediate state to the dashboard. Instead, it keeps the most recent state and broadcasts at a controlled interval.

This is useful for scalability. If many users are connected to the dashboard, broadcasting every event individually could create unnecessary load. Coalescing reduces this load while preserving the final visible state.

10.4 Consistency Window

The average consistency window was approximately 1.07 seconds. For an analytics dashboard, this is acceptable. The dashboard does not need to be strictly consistent at the same moment as the original REST response. It only needs to become consistent shortly afterward.

The measured value shows that the asynchronous pipeline introduces a visible but limited delay. In practice, the system keeps the user-facing API independent from the analytics processing, while still updating the dashboard within a short time interval.

10.5 Bottlenecks

The main bottleneck is not the REST API. The main delay appears in the asynchronous processing path. Possible contributors are:

- Pub/Sub delivery time
- Cloud Function scheduling and execution
- Firestore read/write latency
- second Pub/Sub delivery to the gateway
- WebSocket Gateway coalescing interval

The coalescing interval is intentional. It adds a small controlled delay but reduces WebSocket pressure during bursts.

11 Scalability Discussion

The system is designed around independently deployable components. Service A, the Cloud Function, the WebSocket Gateway, the Dashboard Client and the gRPC Analytics Service can be deployed and scaled separately.

Service A is stateless with respect to analytics and can scale horizontally on Cloud Run. The Cloud Function can also scale automatically based on Pub/Sub events. Firestore is a managed stateful service and can handle concurrent reads and writes without manual server management.

The WebSocket Gateway is more sensitive because WebSocket connections are stateful at the process level. For this project, the gateway was kept at one Cloud Run instance for demo stability. This makes the demo predictable and keeps the connection state simple. In a production system, the gateway would need a shared fanout mechanism or sticky sessions to support multiple gateway instances.

The architecture is therefore scalable in the processing pipeline, while the WebSocket layer would need additional infrastructure for large-scale production use.

12 Resilience Analysis

12.1 Decoupling Through Pub/Sub

Pub/Sub improves resilience because it decouples producers from consumers. Service A does not directly call the Cloud Function or the WebSocket Gateway. It only publishes an event. If the downstream components are temporarily slow, Pub/Sub can retry delivery. This reduces the risk that a problem in the analytics pipeline affects the base movie API.

12.2 Idempotent Event Processing

The Cloud Function uses the `processed-events` collection to avoid processing the same message twice. This is necessary because Pub/Sub may deliver a message more than once.

Without this mechanism, duplicate messages could incorrectly increase the movie view count.

12.3 gRPC Fallback

The WebSocket Gateway uses the gRPC Analytics Service to retrieve top movie statistics. If the gRPC service is unavailable, the gateway can fall back to reading from Firestore. This is a graceful degradation mechanism. The system can continue serving dashboard data even if the internal gRPC path fails.

12.4 WebSocket Reconnection

The dashboard client automatically reconnects when the WebSocket connection is closed. This behavior was tested using the debug endpoint that closes connected clients. After the connection is closed, the dashboard reconnects and loads the current state again.

This mechanism is important because WebSocket connections can be interrupted by network problems, browser behavior or service restarts.

12.5 Protected Debug Endpoints

The debug endpoints are protected using a token header. They are useful for testing and benchmarks because they can reset in-memory metrics or close clients. However, they are not left completely open without control.

13 Comparison with a Real System

A real system that uses similar architectural patterns is **Netflix**. Netflix operates at a much larger scale, but several design ideas are similar to the ones used in this project: microservices, event streams, asynchronous processing, telemetry pipelines and real-time operational monitoring.

At Netflix, user interactions and service events are collected and processed in the background instead of being handled directly in the critical path of the user request. These events can then be used for analytics, personalization, monitoring and operational decisions. This separation allows the main user-facing services to remain responsive while the analytics and monitoring pipelines run independently.

Our project follows the same general pattern at a smaller scale:

- the user-facing API remains synchronous and responsive
- analytics events are processed asynchronously
- an event broker decouples producers and consumers
- aggregated state is stored separately from the base application data
- a dashboard displays operational metrics
- real-time communication is used to show live updates

The comparison can be made more explicit by looking at the main architectural aspects side by side:

Table 7: Comparison between our system and a Netflix-style event-driven architecture

Aspect	Our project	Netflix-style system
User-facing service	Fast Lazy Bee exposes REST endpoints for movie resources and must remain responsive.	User-facing streaming and application services must respond quickly to user actions.
Event generation	A movie access generates a <code>movie_viewed</code> event.	User interactions, logs and operational signals generate large event streams.
Event transport	Pub/Sub decouples Service A from the analytics processor.	Event streaming platforms decouple event producers from downstream consumers.
Asynchronous processing	A Cloud Function processes events and updates the analytics state.	Stream-processing systems process telemetry, monitoring and usage events continuously.
Analytics state	Firestore stores aggregated movie statistics and processed event identifiers.	Large-scale data stores and telemetry platforms store derived metrics and analytics results.
Real-time visibility	The WebSocket Gateway pushes live updates to the dashboard.	Internal dashboards expose real-time operational and business metrics.
Resilience	Pub/Sub retry, idempotency and gRPC fallback reduce the impact of component failures.	Production systems use stronger mechanisms such as multi-region deployment, automated recovery and large-scale observability.

The main similarity is the separation between the synchronous user-facing path and the asynchronous analytics path. In both systems, the user request should not wait for all analytics processing to finish. This keeps the main service responsive, while analytics components can be delayed, retried or scaled independently.

There are also important differences. Netflix uses much more advanced infrastructure, multi-region deployments, large-scale event streaming platforms, complex monitoring systems and production-grade resilience mechanisms. Our implementation applies the same architectural ideas at a smaller scale, using managed cloud services and controlled experiments to validate the main distributed systems concepts.

This comparison shows that the implemented architecture follows patterns used in real distributed systems. Event-driven analytics, asynchronous processing, separate analytics storage and real-time dashboards are common solutions when user-facing services must remain responsive while background processing happens independently.

14 Conclusions

This project implemented a cloud distributed application for real-time analytics. Starting from the Fast Lazy Bee REST API, the system was extended with an event-driven processing pipeline, a Cloud Function, Firestore analytics storage, a WebSocket Gateway, a live dashboard and an internal gRPC analytics service.

A central design decision was to separate the user-facing API from the analytics pipeline. The REST API remains responsible for serving movie resources, while the analytics flow is handled asynchronously in the background. This keeps the normal API response independent from the dashboard processing and makes the system easier to reason about, test and scale.

The final tests showed that the system behaved correctly for the evaluated workloads. All benchmarked HTTP requests succeeded, all generated analytics events were processed, and the measured consistency window was approximately 1.07 seconds. This confirms the expected eventual consistency behavior: the dashboard is updated shortly after the original movie access, not at the exact same moment.

The main trade-off is that the system accepts a short delay in the analytics view in exchange for better decoupling and responsiveness. The performance results also show that most of the delay comes from the asynchronous path, especially under concurrent traffic. The WebSocket Gateway reduces pressure on the realtime layer through update coalescing, while still keeping the visible dashboard state up to date.

Overall, the final system is a functional distributed cloud application with real event processing, persistent analytics state, realtime WebSocket updates, internal gRPC communication, dashboard latency charts and reproducible benchmark tests. The implementation shows that a REST API can be extended into a more complete cloud system by separating responsibilities between services and by using asynchronous communication for the parts that do not need to block the user request.

Use of AI Tools

AI tools were used during the development and documentation process for code review, debugging assistance, benchmark interpretation and report structuring. More specifically, we used **ChatGPT** to help organize implementation ideas, review commands, explain errors, improve wording and structure parts of the report. The generated suggestions were manually checked, adapted and validated through local builds, cloud verification scripts, manual tests and benchmark results. The final implementation, deployment, benchmark execution and interpretation of the results were performed and validated by the team.